FADENA

LICENCIATURA EN CIBERDEFENSA

DISPOSITIVOS REMOTOS E INTERNET DE LAS COSAS
CÁTEDRA: DIEGO RACERO
AÑO: 2026

CARGA HORARIA: 124 HS

PROTOCOLOS IoT

Protocolos Avanzados con implementaciones de seguridad

Protocolos de Comunicación en IoT

Protocolo CoAP (Constrained Application Protocol)

Definición y Filosofía

CoAP es un protocolo de transferencia web especializado para su uso con nodos restringidos y redes restringidas (p. ej., redes de baja potencia y con pérdida). Su filosofía de diseño se puede resumir como **"HTTP para cosas pequeñas"**. Busca replicar el modelo REST (Representational State Transfer) de la web tradicional pero eliminando la sobrecarga del protocolo HTTP y TCP.

Características Técnicas Principales

1. **Arquitectura RESTful:** Utiliza los verbos estándar (GET, POST, PUT, DELETE) para manipular recursos identificados por URIs (`coap://...`). Esto facilita la integración con sistemas web existentes mediante *proxies* de traducción HTTP-CoAP.
2. **Transporte sobre UDP:** A diferencia de HTTP (que usa TCP), CoAP corre por defecto sobre UDP (User Datagram Protocol). Esto reduce drásticamente el *overhead* de conexión y la latencia de establecimiento de enlace.
3. **Fiabilidad Opcional:** Dado que UDP no garantiza la entrega, CoAP implementa su propia capa de fiabilidad ligera mediante dos tipos de mensajes:
    - **Confirmable (CON):** Requiere un acuse de recibo (ACK) del receptor.
    - **Non-confirmable (NON):** "Fire and forget", para datos que se envían frecuentemente (ej. lecturas de temperatura cada segundo).
4. **Observación de Recursos (RFC 7641):** Extiende el modelo cliente-servidor para permitir un patrón de "suscripción". Un cliente puede "observar" un recurso y el servidor enviará notificaciones asíncronas cuando el estado cambie, sin que el cliente tenga que hacer *polling* constante.
5. **Descubrimiento de Recursos:** Soporta el descubrimiento automático de servicios y recursos disponibles en un nodo (generalmente en `/.well-known/core`).

Escenario Ideal

Dispositivos alimentados por batería que necesitan interactuar con servicios web, donde cada byte cuenta. Ejemplo: Medidores inteligentes (Smart Metering) o control de iluminación en edificios inteligentes.

Comunicación WebSockets

Definición y Filosofía

WebSockets es un protocolo que proporciona canales de comunicación **full-duplex** (bidireccional simultánea) sobre una única conexión TCP. Fue diseñado para superar las limitaciones del modelo "pregunta-respuesta" de HTTP en aplicaciones que requieren interactividad en tiempo real.

Funcionamiento Detallado

1. **El Handshake (Apretón de manos):** La conexión inicia siempre como una solicitud HTTP estándar. El cliente envía un *header* `Upgrade: websocket`. Si el servidor acepta, responde con un código `101 Switching Protocols`.
2. **Conexión Persistente:** Una vez establecido el *handshake*, la conexión TCP se mantiene abierta. Ya no existen cabeceras HTTP pesadas en cada mensaje; los datos se envían en tramas (*frames*) ligeras.
3. **Bidireccionalidad Real:** El servidor puede enviar datos al cliente en cualquier momento sin que el cliente lo haya solicitado previamente (Server-Push).
4. **Baja Latencia:** Al eliminar la necesidad de establecer nuevas conexiones TCP y TLS para cada intercambio, la latencia se reduce al mínimo físico de la red.

Diferencia con MQTT/CoAP

WebSockets es un "túnel" de transporte. No define *cómo* debe ser el mensaje (payload), a diferencia de CoAP (que define recursos) o MQTT (que define tópicos Pub/Sub). A menudo, MQTT se encapsula *dentro* de WebSockets para permitir que navegadores web actúen como clientes IoT.

Escenario Ideal

Dashboards de monitoreo en vivo, control remoto de actuadores desde una interfaz web, chats de soporte o cualquier aplicación donde un humano interactúa con el sistema IoT en tiempo real a través de un navegador.

Adaptación MQTT-SN (MQTT for Sensor Networks)

El Problema de MQTT en Redes de Sensores

El MQTT estándar requiere una conexión TCP/IP estable y ordenada. Sin embargo, muchas redes de sensores (como Zigbee, Bluetooth LE o redes industriales propietarias) no soportan

TCP, tienen un MTU (tamaño máximo de paquete) muy pequeño (ej. 127 bytes) o sufren de alta pérdida de paquetes.

Arquitectura y Funcionamiento

MQTT-SN es una variante de MQTT optimizada para redes que no son TCP/IP (generalmente corre sobre UDP o transportes seriales).

1. **Arquitectura con Gateway:** Introduce un componente crítico: el **Gateway MQTT-SN**.
    - Los dispositivos (clientes) hablan MQTT-SN con el Gateway.
    - El Gateway traduce esos mensajes a MQTT estándar y los envía al Broker en la nube.
2. **Optimizaciones de Protocolo:**
    - **Topic ID vs Topic Name:** En lugar de enviar cadenas de texto largas (`"casa/cocina/sensor/temp"`) en cada mensaje, MQTT-SN registra el tópico una vez y le asigna un ID numérico corto (ej. `0x01`). Esto reduce drásticamente el uso de ancho de banda.
    - **Tópicos Predefinidos:** Los IDs de tópicos pueden venir configurados de fábrica, eliminando la fase de registro.
3. **Soporte para "Sleeping Clients":** Permite que un dispositivo se vaya a dormir (modo bajo consumo) y el Gateway almacene los mensajes dirigidos a él. Cuando el dispositivo despierta y hace un "Ping", el Gateway le entrega los mensajes acumulados.
4. **Discovery:** Los clientes pueden descubrir automáticamente la dirección del Gateway en la red mediante broadcast.

Escenario Ideal

Redes de sensores inalámbricas (WSN) masivas sobre Zigbee o LoRa, donde el ancho de banda es extremadamente bajo y los dispositivos duermen el 99% del tiempo para ahorrar batería.

Comparativa y Guía de Selección

Para seleccionar el protocolo adecuado, debemos evaluar las restricciones del hardware y los requisitos del software.

Cuadro Comparativo

Característica	CoAP	WebSockets	MQTT-SN
Modelo	Request/Response (REST)	Bidireccional (Raw socket)	Publish/Subscribe
Transporte	UDP (principalmente)	TCP	UDP, Serial, Non-IP
Complejidad	Media (Manejo de estado)	Baja (Tubería de datos)	Media (Requiere Gateway)
Overhead	Muy Bajo (4 bytes header)	Bajo (tras handshake)	Extremadamente Bajo
QoS (Calidad de Servicio)	Confirmable / Non-Confirmable	Garantizado por TCP	QoS 0, 1, 2 (+ Sleeping)
Idoneidad Web	Requiere Proxy	Nativo en navegadores	No nativo

Guía de Decisión para Proyectos (Heurísticas)

1. Elija CoAP si:

- Necesita compatibilidad con una arquitectura RESTful existente (interacción sencilla con APIs HTTP).
- El dispositivo tiene recursos limitados pero soporta IP (ej. 6LoWPAN).
- La comunicación es esporádica y se tolera cierta pérdida de datos o se prefiere reintentar a nivel de aplicación.

2. Elija WebSockets si:

- El "Cliente" es un navegador web o una aplicación móvil híbrida.
- La red es estable y no hay restricciones severas de energía (dispositivos conectados a la corriente).
- Necesita enviar flujos de datos continuos (streaming de datos) con baja latencia.

3. Elija MQTT-SN si:

- La red física NO soporta TCP/IP (ej. Zigbee puro, Bluetooth mesh sin capa IP).

- El ancho de banda es extremadamente costoso o limitado (satelital, LPWAN).
- Tiene muchos dispositivos que pasan la mayor parte del tiempo "dormidos" y necesitan recibir comandos al despertar.
- Necesita desacoplar completamente al productor de datos del consumidor (modelo Pub/Sub).

Resumen para el Estudiante

- **CoAP** es la web adelgazada para microcontroladores.
- **WebSockets** es el cable directo para aplicaciones web en tiempo real.
- **MQTT-SN** es la mensajería eficiente para redes hostiles y sensores sin IP.

Esta semana realizaremos la actividad obligatoria 2, que se complementa con la actividad obligatoria 1. Tendremos que ver las simulaciones de CoAP, Websockets, pero también volveremos sobre MQTT. Esta vez tendremos que usar un broker público de MQTT, el dispositivo simulado y un cliente web de HiveMQ para ver los resultados del sensor simulado.

Como siempre tendremos que realizar análisis con WireShark